Lab 13: AI APIs as Data Tools

INFO 4617/5617 · Week 13 · Chapter 13, AI APIs as Data Tools

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

Monday and Wednesday work on the City of Boulder's revenue reports from week 9, the PDFs whose last page is a picture of a table. Friday moves to Wikipedia:

- **Part 1, Monday: document understanding**, reading a whole PDF, pictures of tables included. Datalab's API turns it into Markdown a program can read, and we check what it read;
- **Part 2, Wednesday: a model as a function**. Through OpenRouter, a language model reads one page and returns JSON in a shape we define, and we check its numbers against week 9's;
- **Part 3, Friday: search by meaning**. Embeddings, lists of numbers that stand for what a passage is about, measure how the opening paragraphs of five Wikipedia articles about politicians have changed since 2004, and find the versions that answer a question. A model answers from those versions and names its sources, or says it can't answer. Last, **tools**: a model reads the tools that the Census Bureau's MCP server offers, and writes one request to week 11's API, which our code sends and checks against week 11's figure.

No chatbots: every call to a model is one request in and one JSON answer out, and every answer gets checked. Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. You need **two free keys**, and Friday also needs week 11's, each stored as an environment variable by one of week 11's three safe ways (set it, close the terminal, open a new one, and start Jupyter from there):

- `DATALAB_KEY`: sign up at datalab.to, then copy a key from API Keys. The free plan includes a monthly allowance of at least $10 (pricing); this lab spends about 9 cents of it;
- `OPENROUTER_KEY`: sign up at openrouter.ai, then create a key at Keys. The models we use end in `:free`: they cost nothing, up to a daily limit;
- `CENSUS_KEY`, for Friday's Step 14: week 11's key from the Census Bureau. If week 11's lab could read it, so can this one.

Treat each key like a password: never paste it into a cell or a code file. On Wednesday and Friday, start at that day's setup step, or run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Parts 1 and 2 were written on 2026-11-07 from the chapter 13 plan, and Part 3 was rewritten on 2026-11-09, when the instructor moved Friday to Wikipedia leads and the Census Bureau's MCP server. The same day the whole key was revised after an audit against the live APIs, Datalab's and OpenRouter's documentation, and the chapter (issue #165). It follows chapter 13 as merged in the textbook (commit 3036573, PR #309): Step 10 adds the schema to the instruction, as "Seven Reports" does, and Step 11's given cleanup repeats `lead_text()`. Friday's data, read from `wiki` and `cmcp`, are the chapter's, copied to `labs/week-13/data/`. Run on 2026-11-09 with `labs/tools/run_key.py`. Every cell has output and none is tagged `not-run-live`, but two steps' outputs come from other runs that day, of the same code: Wikimedia refused the cloud machine at Step 11's `action=parse`, so Step 11's four cells keep an earlier run's outputs; and one answer in Step 10's first loop came back with no content (`None`), so that loop's table is from a rerun of Part 2 soon after. A rerun should match Datalab's outputs and costs (4.5 cents a report, with `skip_cache`), Step 11's lead (the same text as the saved one), the embedding scores, and the Census figure, $103,994 with a margin of error of $2,268. It won't necessarily match the free models: which model answers, and what it gets wrong, change from run to run. Here `nvidia/nemotron-3.5-super-vl:free` answered every request but the microsprint's

to Dots. In Step 10's first loop it matched both totals for 4 of 7 reports, and for 2021 returned figures that aren't on the page; with the schema in the instruction, 7 of 7. On 2026-11-10 Step 8 and Step 14 were changed after the audit's second pass (Step 8 prints the response's `error` and whether the model that answered is in `MODELS`; Step 14's given cell reads `args` itself), and those cells, with Step 9's comparison, were run again on their own. In Step 14 the model then asked for the whole table, `group` B19013 (the run of 2026-11-09 asked for `B19013_001E` alone, without the margin of error), and in the microsprint it chose `resolve-geography-fips`, which our code doesn't run.

Part 1 — Monday: from a PDF to text a program can read

In week 9, pdfplumber read the text layer of Boulder's revenue reports exactly. It stopped at Exhibit 3 of the December 2024 report, a picture of a table, beside a note asking readers to phone the city for "a screen readable version". Today a **document-understanding** API, one that reads a page's pictures as well as its text, reads the whole report. Then we check what it read, the way week 9 checked pdfplumber.

Step 0 — Set up, and check your keys (5 minutes)

Import what we need, define the User-Agent, and check that both keys are set before reading them, as week 11 did. If one says `MISSING`, quit Jupyter, open a new terminal, and start Jupyter again from there. If you can't restart now, run the given cell: it asks for a missing key with `getpass()`, which doesn't show what you type, and does nothing if both keys are set. The chapter's "Two Keys" has the steps to get them.

```
# Import os, re, time, requests, and pandas (as pd); define HEADERS as in week 12, with your email
# Print whether "DATALAB_KEY" and "OPENROUTER_KEY" are set or MISSING, without printing either key
import os
import re
import time
import requests
import pandas as pd

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("DATALAB_KEY:", "set" if os.environ.get("DATALAB_KEY") else "MISSING")
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
```

```
DATALAB_KEY: set
OPENROUTER_KEY: set
```

```
# Given: only for a key the check said was MISSING, ask for it with getpass. If both said set, this does nothing
from getpass import getpass
for name in ["DATALAB_KEY", "OPENROUTER_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
```

```
# Read DATALAB_KEY into datalab_key; datalab_base = "https://www.datalab.to/api/v1"
# book: the address of week 9's saved PDFs, "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-09/"
datalab_key = os.environ["DATALAB_KEY"]
datalab_base = "https://www.datalab.to/api/v1"
book = "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-09/"
```

💬 **Discuss**

- Census, FRED and YouTube gave you keys to read their data; Datalab and OpenRouter read whatever you send them. What wouldn't you send?
- Datalab bills by the page, after a free allowance. What does that allowance let the company learn about you?

Step 1 — Submit a document (8 minutes)

Datalab's Convert API takes a PDF and returns Markdown. It takes the PDF in one of two fields: `file_url` sends an address, and Datalab downloads the PDF from there; `file` uploads a PDF from your computer, which `requests` sends as `files={"file": (name, open(path, "rb"), "application/pdf")}`. We send the address of the book's saved copy of the December 2024 report, so we use `file_url`, and every field is text in `data=`. We ask for `markdown` in `balanced` mode, and set `paginate` so that each page is marked. Datalab keeps a copy of what it converts, and hands that copy back, free, when the same file comes again. So we also set `skip_cache`: Datalab converts the file afresh, and the cost we see is the real one. The key goes in an `X-API-Key` header. The answer isn't the document. Converting takes seconds, so the API answers at once with a **ticket**, `request_check_url`: an address where the result will be when it's ready. See the chapter's "Submitting a Document".

```
# data: "file_url", the PDF's address, book + "revenue-report-2024-12.pdf" (not "file", which
 ↪  uploads a file);
#   output_format "markdown", mode "balanced", paginate "true", skip_cache "true"
# job: the JSON of a POST to f"{datalab_base}/convert" with data= and headers {"X-API-Key":
 ↪  datalab_key} plus HEADERS; print it
data = {"file_url": book + "revenue-report-2024-12.pdf", "output_format": "markdown",
        "mode": "balanced", "paginate": "true", "skip_cache": "true"}
job = requests.post(f"{datalab_base}/convert", data=data,
                    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(job)
```

```
{'success': True, 'error': None, 'request_id': 'GgNXssQIPMoMUkTtY8uryA', 'request_check_url':
 ↪  'https://www.datalab.to/api/v1/convert/GgNXssQIPMoMUkTtY8uryA', 'versions': None,
 ↪  'checkpoint_reused': None}
```

💬 **Discuss**

- Every API so far answered with the data. Why might a service answer with a ticket instead?
- We sent an address, not a file. When would you have to upload the file instead?
- Typed as `"file": "revenue-report-2024-12.pdf"`, this request got HTTP 422 with `'loc': ['body', 'file']` and `'msg': "Value error, Expected UploadFile, received: <class 'str'>"`. What do `loc` and `msg` say went wrong, and why is the first line to fail Step 2's `job["request_check_url"]`?

Step 2 — Wait for the result (10 minutes)

Ask the ticket's address whether the job is done. The result check answers with a `status`: `processing` while Datalab works, then `complete` or `failed`. Asking again and again until a job is done is called **polling**. Take it in two moves: ask once by hand and print the status; then a given loop asks every two seconds while the status is `processing`, the way week 12's loop followed the `cursor` until there wasn't one. It waits only while `processing`, not until `complete`, because a `failed` job never becomes `complete`, and a loop waiting for that would never end. Then it prints `error`, which is empty when the conversion worked. Read it even when the status is `complete`: a job for an address with no PDF behind it came back `complete`, with `success` set to `False` and a 404 in `error`. The finished answer says how many pages it read, `page_count`, and what they cost, `total_cost`, in cents. (For a large document, it can hold a `result_url`, an address to download the result from, instead of the `markdown` itself.)

```
# check: the JSON of a GET to job["request_check_url"] with the same headers; print its "status"
check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
```

```
print(check["status"])
```

```
processing
```

```
# Given: ask again every two seconds while the job is processing; then print its status and
 ↪  error, and what it read and cost
while check["status"] == "processing":
    time.sleep(2)
    check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
print("status:", check["status"], "error:", check.get("error"))
print(check["page_count"], "pages,", check["total_cost"], "cents,", round(check["runtime"], 1),
 ↪  "seconds of processing")
```

```
status: complete error:
15 pages, 4.5 cents, 14.8 seconds of processing
```

💬 Discuss

- The loop stops on `failed` as well as `complete`, and prints the `error`. If a job stayed `processing` and never finished, what would your notebook do, and how does the chapter's loop guard against that?
- Fifteen pages cost a few cents. About what would every year-end report Boulder has published cost?
- When would you choose `lite` mode, which cost the chapter about an eighth as much for this report?

Step 3 — Read the Markdown, page by page (12 minutes)

The document is in `check["markdown"]`, one long string. With `paginate`, each page starts with a **page marker**: a number in braces, such as `{14}`, then a line of dashes. In the pattern we split on, `\{\d+\}` matches the number in its braces and `-{10,}` ten or more dashes. So `re.split()` on that pattern gives a list of pages; drop the empty piece before the first marker. Python counts from 0, so page 2 is `pages[1]`. Print page 2, the summary table week 9 read with pdfplumber; then page 15, Exhibit 3, the picture of a table. **Don't print page 1:** the cover gives a staff member's email and phone number, and we keep contact details out of our outputs, as week 9 blacked them out. For the same reason, print Exhibit 3 from its table, which starts at the first `|`, below a note with the sales tax office's phone and email. Its lines are long, so print the first 150 characters of each. See the chapter's "Reading the Markdown".

```
# pages: re.split(r"\n*\{\d+\}-{10,}\n*", markdown) without its first, empty piece; print how
 ↪  many pages
# Print the first 1,200 characters of pages[1], the summary; then of pages[14], Exhibit 3. Never
 ↪  pages[0]
markdown = check["markdown"]
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
print(len(pages), "pages")
print(pages[1][:1200])
```

```
15 pages
This report provides information and analysis related to 2024 unaudited taxes and fees including
 ↪  sales and use tax, accommodations tax, admissions tax, disposable bag fee, electronic smoking
 ↪  device tax, food service tax, marijuana tax, short-term rental tax, sugar sweetened beverage
 ↪  tax, and the trash tax. The revenue reported below was received by the city through January
 ↪  2025 and is largely attributed to economic activity through the month of December. Details by
 ↪  industry, geographic zone and month are found on pages 3 through 7 of this report.

## **Sales and Use Tax Summary**
```

```
| <b>Sources - Related to Current Economic Activity</b>                | <b>December YTD</b>  |
 ↪  |                      |                      |                          |
|-----------------------------------------------------------------------|----------------------|--┐
 ↪  --------------------|--------------------|--------------------|--------------------------|
|                                                                       | <b>2023</b>          |
 ↪  <b>2024</b>          | <b>\$ Change</b>    | <b>YTD % Change</b> | <b>12 Month % Change</b> |
| Sales Tax
```

```
# exhibit: pages[14] from its first "|" (the table, below the note); print the first 150
 ↪  characters of its first 10 lines
exhibit = pages[14][pages[14].index("|"):]
for line in exhibit.splitlines()[:10]:
    print(line[:150])
```

```
| REVENUE<br>CATEGORY            | YEAR        | JAN               | FEB               | MAR
 ↪  | APR               | MAY               | J
|--------------------------------|-------------|-------------------|-------------------|--------┐
 ↪  -----------|-------------------|-------------------|--
|                                |             |                   |                   |
 ↪  |                   |                   |
| <b>Retail Sales Tax</b>        | 2019        | 8,590,452         | 7,489,624         |
 ↪  9,606,398         | 8,214,047         | 8,832,480         | 1
|                                | 2020        | 7,761,028         | 7,370,943         |
 ↪  10,025,017        | 6,090,136         | 7,059,371         | 9
|                                | 2021        | 8,059,343         | 7,608,759         |
 ↪  10,351,245        | 8,666,637         | 9,229,065         | 1
|                                | 2022        | 9,592,143         | 9,009,882         |
 ↪  11,376,046        | 11,242,614        | 10,608,781        | 1
|                                | 2023        | 10,246,130        | 9,156,111         |
 ↪  12,665,356        | 10,465,614        | 10,686,280        | 1
|                                | <b>2024</b> | <b>10,556,772</b> | <b>9,223,353</b>  |
 ↪  <b>12,642,682</b> | <b>11,464,301</b> | <b>10,743,033</b> | <
| Change from prior year (month) |             | 3.03%             | 0.73%             | -0.18%
 ↪  | 9.54%             | 0.53%             | 2
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Datalab doesn't only read text: where a page has a chart, it writes a description of it. Chart 8, on page 9, is a pie chart of marijuana revenue, with its figures printed on the slices.

Do: print page 9 (`pages[8]`), and compare the description under the chart with the figures in its table. **Share:** one sentence of the description that the figures contradict, and who wrote that sentence: the city, or Datalab?

```
# Microsprint: print pages[8]; read the description under Chart 8 against its figures
print(pages[8][:2200])
```

```
### Marijuana Tax Revenues

The City of Boulder collects revenue from the sale of both medical and recreational marijuana.
 ↪  The city collects the base 3.86% sales and use tax on medical and recreational marijuana. The
 ↪  city also collects an additional 3.50% sales and use tax on recreational marijuana, and a
 ↪  5.0% excise tax when marijuana is transferred from a cultivation facility to a dispensary,
 ↪  testing facility or facility that produces marijuana infused products. In addition, the city
 ↪  receives a revenue share back from state recreational marijuana taxes, which is reflected in
 ↪  Chart 8 below.
```

```
**Chart 8: Distribution of YTD Marijuana Revenues**

![Pie chart: Breakdown of marijuana revenue by category and
 ↪  amount.](bedcca5cdf168e3508ef511d94ec514c_img.jpg)

| Category                                                | Amount      |
|---------------------------------------------------------|-------------|
| State Share-Back                                        | \$529,220   |
| Medical Marijuana (3.86% Base)                          | \$52,937    |
| Recreational Marijuana (3.86% Base)                     | \$1,262,688 |
| Recreational Marijuana (3.5% Additional + 5.0% Excise)* | \$1,341,007 |

A pie chart showing the distribution of marijuana revenue. The largest slice is Recreational
 ↪  Marijuana (3.86% Base) at \$1,262,688, followed by Recreational Marijuana (3.5% Additional +
 ↪  5.0% Excise)\* at \$1,341,007. State Share-Back is \$529,220, and Medical Marijuana (3.86%
 ↪  Base) is \$52,937.

Pie chart: Breakdown of marijuana revenue by category and amount.

\*Dedicated to the General Fund.

**Chart 9: Historical Trends for YTD Marijuana Revenues<sup>3</sup>**

![Line chart: Comparison of Medical Marijuana, Rec Marijuana, and Grand Total from 2021 to
 ↪  2024.](0f3e3ea50bcceb86f6c524ab2b6f3e7a_img.jpg)

A line chart showing the financial performance of Medical Marijuana, Rec Marijuana, and Grand
 ↪  Total from 2021 to 2024. All three categories show a downward trend over the four-year
 ↪  period.

| Year | Medical Marijuana | Rec Marijuana | Grand Total |
|------|-------------------|---------------|-------------|
| 2021 | \$208,408         | \$5,429,871   | \$5,638,279 |
| 2022 | \$18
```

💬 Discuss

- The city's note asks readers to phone for a screen readable version of Exhibit 3. Who would have to check Datalab's table before the city could publish it as that version?
- The description of Chart 8 calls one slice the largest, then gives a larger one. How would you tell what a model wrote on these pages from what it read off them?
- Page 10's bar chart of sugar-sweetened beverage tax has no figures printed on it. Print `pages[9]`: where could the numbers Datalab wrote beside it have come from?

Step 4 — Check what it read (12 minutes)

Week 9's first check was a sum: a row's months should add up to its total (chapter 9, "Checking an Extraction"). Exhibit 3 has 30 rows of figures, one per tax and year. In the Markdown, each row of the table is one line, with its cells between `|` characters. Split each line on `|`, and strip the `<b>` tags and the commas from its cells. Where `cells[2]` is a year, add the twelve months and compare the sum with the total. The report rounds each figure to the dollar, so allow $5, as week 9 did.

```
# For each line of pages[14]: cells = its pieces split on "|", stripped, without "<b>", "</b>"
 ↪  and ","
# Keep a line whose cells[2] is a year (re.fullmatch(r"20\d\d", ...)); add the 12 months,
 ↪  cells[3:15], and compare with cells[15]
# Count the rows checked, and print any whose months miss the total by more than $5
```

```
checked = 0
for line in pages[14].splitlines():
    cells = [c.strip().replace("<b>", "").replace("</b>", "").replace(",", "") for c in
 ↪  line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        months = sum(int(c) for c in cells[3:15])
        checked += 1
        if abs(months - int(cells[15])) > 5:
            print(cells[2], months, cells[15])
print(checked, "rows checked")
```

```
30 rows checked
```

⏱️ Microsprint (5 minutes, alone or in pairs)

The 2018 report is older: its Exhibit 3 is a table of text, not a picture.

Do: submit `revenue-report-2018.pdf` the same way (Steps 1 and 2), and run Step 4's check on its page 15, `pages_2018[14]`. **Share:** its page count and cost, how many rows the check found, and how many missed.

```
# Microsprint: set data["file_url"] to the 2018 report; submit, wait (the loop again), split into
 ↪  pages_2018, and check pages_2018[14]
data["file_url"] = book + "revenue-report-2018.pdf"
job = requests.post(f"{datalab_base}/convert", data=data, headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
while check["status"] == "processing":
    time.sleep(2)
    check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
print("status:", check["status"], "error:", check.get("error"))
print(check["page_count"], "pages,", check["total_cost"], "cents")
pages_2018 = re.split(r"\n*\{\d+\}-{10,}\n*", check["markdown"])[1:]
checked = 0
for line in pages_2018[14].splitlines():
    cells = [c.strip().replace("<b>", "").replace("</b>", "").replace(",", "") for c in
 ↪  line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        checked += 1
        if abs(sum(int(c) for c in cells[3:15]) - int(cells[15])) > 5:
            print(cells[2])
print(checked, "rows checked")
```

```
status: complete error:
15 pages, 4.5 cents
28 rows checked
```

💬 Discuss

- Every row added up. What can a sum not catch? (Week 9's Docling put the right figures beside the wrong label.)
- Week 9 ran Docling on a laptop, free, in about a minute; Datalab took seconds and cost cents. What else differs between the two if the PDF holds a student's records rather than a city's?
- Our code assumes every request worked. If Datalab answered 401 (Unauthorized) because the key was wrong, which line would fail first?

Step 5 — Save the text for Wednesday (3 minutes)

Write the 2024 report's Markdown to a file, so Wednesday doesn't have to pay to read it again. (Wednesday also works from the book's saved copies of all seven year-end reports, made the same way on 2026-11-07, so a student who missed today can start there.)

```
# Write markdown, the 2024 report's, to "revenue-report-2024-12.md" with open(..., "w",
 ↪  encoding="utf-8")
with open("revenue-report-2024-12.md", "w", encoding="utf-8") as f:
    f.write(markdown)
```

End of Part 1

On Wednesday, Part 2 hands one page of this Markdown to a language model and asks for JSON in a shape we define: the summary table's totals. Then we check the model's numbers against week 9's, for seven years of reports. It starts with its own setup step, so you don't need today's cells to have run.

Part 2 — Wednesday: a model as a function

A **language model**, the kind of program behind a chatbot, can be called like any other API: we send it an instruction and one page, and it returns an answer. We won't chat with it. We ask for **structured output**, JSON in a shape we define, so the answer goes straight into a DataFrame. Then we check it, because a model's numbers look right whether or not they are.

Step 6 — Set up again (5 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Import what we need, check `OPENROUTER_KEY`, and set two addresses: OpenRouter's API, and the book's saved copies of Datalab's Markdown for the seven year-end reports (`data/ch-13/`), made as on Monday. OpenRouter takes one kind of request and passes it to models from many companies. Models whose ID ends in `:free` cost nothing, but they're shared, so they're sometimes busy. Then run the given cell under it, whatever the check said: it asks for the key if it's missing, and reads it.

```
# Import os, re, json, time, requests, and pandas (as pd); define HEADERS; check OPENROUTER_KEY
 ↪  (don't read it yet)
# openrouter_base = "https://openrouter.ai/api/v1"; saved = the book's data/ch-13/ address, as
 ↪  book was Monday's data/ch-09/
import os
import re
import json
import time
import requests
import pandas as pd

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
openrouter_base = "https://openrouter.ai/api/v1"
saved = "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-13/"
```

```
OPENROUTER_KEY: set
```

```
# Given: run this whatever the check said. If OPENROUTER_KEY was MISSING, it asks for it with
 ↪  getpass, in a box
# under this cell; then, either way, it reads the key into openrouter_key
from getpass import getpass
```

```
for name in ["OPENROUTER_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
openrouter_key = os.environ["OPENROUTER_KEY"]
```

💬 Discuss

- OpenRouter stands between you and the companies that run the models. What does that make easier?
- With OpenRouter in the middle, who sees each page you send?

Step 7 — Find the summary page (5 minutes)

Read the 2024 report's Markdown from the saved copy, split it into pages as on Monday, and keep the first page that holds the table's last row, `Total Sales and Use Tax`. That one page is all the model will see. It's about 1,500 **tokens**. A token is a word or a piece of one: the unit a model reads and writes text in, and is billed by.

```
# markdown: the .text of a GET to saved + "revenue-report-2024-12.md"; pages: split as on Monday
# summary: the first page that contains "Total Sales and Use Tax"; print its first 1,000
 ↪  characters
markdown = requests.get(saved + "revenue-report-2024-12.md", headers=HEADERS).text
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
summary = [page for page in pages if "Total Sales and Use Tax" in page][0]
print(summary[:1000])
```

```
This report provides information and analysis related to 2024 unaudited taxes and fees including
 ↪  sales and use tax, accommodations tax, admissions tax, disposable bag fee, electronic smoking
 ↪  device tax, food service tax, marijuana tax, short-term rental tax, sugar sweetened beverage
 ↪  tax, and the trash tax. The revenue reported below was received by the city through January
 ↪  2025 and is largely attributed to economic activity through the month of December. Details by
 ↪  industry, geographic zone and month are found on pages 3 through 7 of this report.

## **Sales and Use Tax Summary**

| <b>Sources - Related to Current Economic Activity</b>               | <b>December YTD</b>  |
 ↪  |                     |                     |                          |
|---------------------------------------------------------------------|----------------------|--┘
 ↪  --------------------|---------------------|---------------------|--------------------------|
|
```

💬 Discuss

- The phrase is on two pages of every report, and we took the first. What would the model see if that page held a different table?
- What would the model miss if the table ran onto the next page?

Step 8 — One request, one JSON answer (12 minutes)

A request to chat completions holds a list of `messages`: a `system` message with the instruction, then a `user` message with the page. That's the whole conversation; we never add a third message. `response_format` asks for structured outputs. With `type` `json_schema`, the model must answer in the shape of the given `SCHEMA`, a **JSON Schema**: a description, itself in JSON, of the fields an answer must have and the type of each. **Temperature** sets how much chance goes into each word the model picks; `temperature` 0 asks for its most likely answer. We send a **fallback list** of free `models` rather than one model: if the first is busy, OpenRouter tries the next (model fallbacks). Our `MODELS` list holds three free models, in this order: `nvidia/nemotron-3.5-super-vl:free`, `google/gemma-4-31b-it:free`,

and `dots-studio/dots-3-note-preview:free` (checked on 2026-11-07; free models come and go, so check OpenRouter's list before class). Not every provider that runs a model supports structured outputs, though. On 2026-11-09, the only free provider of `google/gemma-4-31b-it:free` offered plain JSON but not a schema, and OpenRouter can send a request to such an endpoint anyway, where the schema isn't enforced. So the body also sets `provider` to `{"require_parameters": True}`, which tells OpenRouter to skip any endpoint that doesn't support every parameter we send (provider routing). `models`, with an s, takes the list; an embeddings request, on Friday, takes one name under `model`. Mind which: with `model` here, or with neither, OpenRouter doesn't refuse the request. It answers from your account's default model, which may not be free. So before reading the answer, print two things. `response.get("error")` is OpenRouter's own message when a request fails, and `None` when it worked: without it, every failure looks the same, a `KeyError` on the next line. And `response["model"] in MODELS` says whether the model that answered is one we asked for. The answer's text is in `choices[0]["message"]["content"]`, a JSON string, and `json.loads()` turns it into a dictionary. The response's `usage` counts the tokens the request used. Among them are `reasoning_tokens`: text some models write to work an answer out before they give it, which isn't part of the answer. See the chapter's "One Request, One JSON Object".

```
# Given: the shape of the answer, as a JSON Schema: four fields, each with a type and a
 ↪  description
MONTHS = ["January", "February", "March", "April", "May", "June", "July",
          "August", "September", "October", "November", "December"]
SCHEMA = {
    "type": "object",
    "properties": {
        "month": {"type": "string", "enum": MONTHS,
                  "description": "The month the year-to-date figures run through"},
        "year": {"type": "integer",
                 "description": "The report's current year: the later of the two years the table
                  ↪  compares"},
        "total_prior": {"type": "integer",
                        "description": "The table's last row, Total Sales and Use Tax, for the
                         ↪  earlier year, in dollars"},
        "total_current": {"type": "integer",
                          "description": "The table's last row, Total Sales and Use Tax, for the
                           ↪  report's current year, in dollars"},
    },
    "required": ["month", "year", "total_prior", "total_current"],
    "additionalProperties": False,
}
INSTRUCTION = ("You extract figures from one page of a City of Boulder revenue report. "
               "Copy numbers exactly as printed, without $ or commas. Use only the page.")
```

```
# MODELS: three free models, in order; body: models, temperature 0, messages (system INSTRUCTION,
 ↪  user summary),
# response_format {"type": "json_schema", "json_schema": {"name": "totals", "strict": True,
 ↪  "schema": SCHEMA}},
# and provider {"require_parameters": True}; response: the JSON of a POST to
 ↪  f"{openrouter_base}/chat/completions"
# with json=body and an Authorization header. Print response.get("error"), then "model" and
 ↪  whether it's in MODELS, and "usage";
# answer: json.loads() of the first choice's content; print it
MODELS = ["nvidia/nemotron-3.5-super-vl:free", "google/gemma-4-31b-it:free",
 ↪  "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
        "messages": [{"role": "system", "content": INSTRUCTION}, {"role": "user", "content":
         ↪  summary}],
        "response_format": {"type": "json_schema", "json_schema": {"name": "totals", "strict":
         ↪  True, "schema": SCHEMA}},
```

```
        "provider": {"require_parameters": True}}
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response.get("error"))
print(response["model"], response["model"] in MODELS)
print(response["usage"])
answer = json.loads(response["choices"][0]["message"]["content"])
print(answer)
```

```
None
nvidia/nemotron-3.5-super-vl:free True
{'prompt_tokens': 1568, 'completion_tokens': 2996, 'total_tokens': 4564, 'cost': 0, 'is_byok':
 ↪  False, 'prompt_tokens_details': {'cached_tokens': 0, 'cache_write_tokens': 0, 'audio_tokens':
 ↪  0, 'video_tokens': 0}, 'cost_details': {'upstream_inference_cost': 0,
 ↪  'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0},
 ↪  'completion_tokens_details': {'reasoning_tokens': 2943, 'image_tokens': 0, 'audio_tokens':
 ↪  0}}
{'month': 'December', 'year': 2025, 'total_prior': 175517129, 'total_current': 173908431}
```

💬 Discuss

- If a different model answered your neighbor, can you compare your two results?
- Your `usage` shows how many `reasoning_tokens` the model wrote before its answer. Why would they matter on a paid model?
- The free models answer 429, Too Many Requests, when they're busy. If all three did, what would `response.get("error")` print, and which line of this cell would fail?
- The list also passes a request on when the first model refuses it, for a misspelled `"role"`, say. If Dots answered and you expected Nemotron, how would you tell a busy model from a mistake in your `body`?

Step 9 — Check it against week 9 (8 minutes)

Week 9 read the same table with pdfplumber, by position, and checked its sums, so we trust its figures. They're our **ground truth**: the answers we check others against. Compare each of the model's fields with them: the two totals, the year, and the month. A model that copies numbers well can still misread which year is which.

```
# Given: week 9's figures for the last row, Total Sales and Use Tax, read with pdfplumber
 ↪  (chapter 9, "The Time Series"):
# for each report, the year before and the report's own year
TRUTH = {
    "revenue-report-2018": (134538892, 140648710), "revenue-report-2019": (140609751, 141954650),
    "revenue-report-2020": (141954650, 135195413), "revenue-report-2021-12": (135195413,
     ↪  152302653),
    "revenue-report-2022-12": (152302653, 169112615), "revenue-report-2023-12": (169112615,
     ↪  175517129),
    "revenue-report-2024-12": (175517129, 173908431),
}
```

```
# prior, current: TRUTH["revenue-report-2024-12"]; print, for each total, the model's figure,
 ↪  week 9's, and whether they match
# Then print the model's year and month: are they 2024 and December?
prior, current = TRUTH["revenue-report-2024-12"]
print("prior:  ", answer["total_prior"], prior, answer["total_prior"] == prior)
print("current:", answer["total_current"], current, answer["total_current"] == current)
print(answer["year"], answer["month"])
```

```
prior:   175517129 175517129 True
current: 173908431 173908431 True
2025 December
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: send `body` to Dots, the third model in `MODELS`, on its own: `{**body, "models": [MODELS[2]]}` copies `body`'s keys, as `**HEADERS` does in `headers`, and replaces its list of models with Dots alone. Print the model that answered and its answer. (Gemma, the second, can't take this request: sent alone with `require_parameters`, it answers 404, "No endpoints found that can handle the requested parameters".)
Share: did Dots agree with Step 8's answer from Nemotron, and with week 9? Which field differed, if any?

```
# Microsprint: send {**body, "models": [MODELS[2]]}, body with Dots alone in its list; print the
 ↪  model that answered and its answer
response = requests.post(f"{openrouter_base}/chat/completions", json={**body, "models":
 ↪  [MODELS[2]]},
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
```

```
dots-studio/dots-3-note-preview:free {'month': 'December', 'total_current': 171006199,
 ↪  'total_prior': 172401668, 'year': 2024}
```

💬 Discuss

- If Dots and Nemotron disagreed, which was right, and which disagreement would you have caught without week 9's figures? If they agreed, does that make them right?
- `temperature` 0 asks for the most likely answer. Why might your answer differ from your neighbor's, or from one day to the next, even when the same model gives it?
- If the model had been wrong and you hadn't had week 9's figures, how would you have found out?

Step 10 — Seven reports (20 minutes)

Now the same request for every year-end report from 2018 to 2024. We'll send it seven times, so gather Step 8's lines into a function, `extract_totals(page)`, that takes a page and returns the answer dictionary and the model that gave it. Then loop over `TRUTH`'s reports. For each, read its Markdown, find its summary page as in Step 7, call `extract_totals()`, and compare the answer with week 9's. Pause two seconds between requests. Then add the schema to the instruction, as text, as the chapter does: `json.dumps()` writes `SCHEMA` as a string of JSON, which can join the instruction's words. `extract_totals()` reads `INSTRUCTION` each time it runs, so the given cell under that line, the same loop again, unchanged, sends the new instruction. Compare its table with the first. See the chapter's "Seven Reports".

```
# extract_totals(page): Step 8's body, provider and all, and its request, with the page as the
 ↪  user message;
# return the answer and response["model"]
def extract_totals(page):
    body = {"models": MODELS, "temperature": 0,
            "messages": [{"role": "system", "content": INSTRUCTION}, {"role": "user", "content":
             ↪  page}],
            "response_format": {"type": "json_schema", "json_schema": {"name": "totals",
             ↪  "strict": True, "schema": SCHEMA}},
            "provider": {"require_parameters": True}}
    response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                             headers={"Authorization": f"Bearer {openrouter_key}",
 ↪  **HEADERS}).json()
    return json.loads(response["choices"][0]["message"]["content"]), response["model"]
```

```
# rows: for each report in TRUTH, read its Markdown, find the summary page, extract_totals() it,
 ↪  and add the answer,
# the model, and whether each total matches week 9; sleep 2 seconds between reports. results: a
 ↪  DataFrame; print it
rows = []
for report, (prior, current) in TRUTH.items():
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", requests.get(saved + report + ".md",
 ↪  headers=HEADERS).text)[1:]
    summary = [page for page in pages if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
    rows.append({"report": report, "model": model, **answer,
                 "prior_ok": answer["total_prior"] == prior, "current_ok":
                  ↪  answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
print(results.drop(columns="model").to_string())
print(results["model"].value_counts())
```

```
                   report     month  year  total_prior  total_current  prior_ok  current_ok
0     revenue-report-2018  December  2018    134538892      139331838      True       False
1     revenue-report-2019   January  2020    141954650      139548151     False       False
2     revenue-report-2020   January  2021    141954650      135195413      True        True
3  revenue-report-2021-12   January  2025      1000000        1200000     False       False
4  revenue-report-2022-12  December  2022    152302653      169112615      True        True
5  revenue-report-2023-12  December  2023    169112615      175517129      True        True
6  revenue-report-2024-12  December  2025    175517129      173908431      True        True
model
nvidia/nemotron-3.5-super-vl:free    7
Name: count, dtype: int64
```

```
# INSTRUCTION: itself, then " Answer with JSON that fits this schema: ", then SCHEMA as text,
 ↪  with json.dumps()
INSTRUCTION = INSTRUCTION + " Answer with JSON that fits this schema: " + json.dumps(SCHEMA)
```

```
# Given: the loop above again, unchanged, now with the schema in INSTRUCTION; compare its table
 ↪  with the first
rows = []
for report, (prior, current) in TRUTH.items():
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", requests.get(saved + report + ".md",
 ↪  headers=HEADERS).text)[1:]
    summary = [page for page in pages if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
    rows.append({"report": report, "model": model, **answer,
                 "prior_ok": answer["total_prior"] == prior, "current_ok":
                  ↪  answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
print(results.drop(columns="model").to_string())
print(results["model"].value_counts())
```

```
                   report     month  year  total_prior  total_current  prior_ok  current_ok
0     revenue-report-2018  December  2018    134538892      140648710      True        True
1     revenue-report-2019  December  2019    140609751      141954650      True        True
2     revenue-report-2020  December  2020    141954650      135195413      True        True
3  revenue-report-2021-12  December  2021    135195413      152302653      True        True
```

```
4  revenue-report-2022-12  December  2022    152302653      169112615      True        True
5  revenue-report-2023-12  December  2023    169112615      175517129      True        True
6  revenue-report-2024-12  December  2024    175517129      173908431      True        True
model
nvidia/nemotron-3.5-super-vl:free    7
Name: count, dtype: int64
```

💬 **Discuss**

- Where on the page did a wrong report's figures come from, in the first table? (Print its summary page and read the table's last two rows.)
- The schema was in `response_format` from the start. Why would sending it again, as text in the instruction, change the answers, and would you still check them against week 9?
- In the key's run, the 2021 report came back with 1,000,000 and 1,200,000, round figures that appear nowhere on its page. Would a sum check have caught them? What did?
- The loop has no error handling. If the fifth request got a 429 from every model, or (as happened once in the key's run) an answer with no content, so that `json.loads(None)` raised a `TypeError`, what would `rows` hold when the cell stopped?

End of Part 2

On Friday, Part 3 leaves the revenue reports for Wikipedia. Embeddings measure how the opening paragraphs of five politicians' articles have changed since 2004, and find the versions that answer a question. A model answers from them, with the IDs of its sources, or says it can't. Then a model reads the tools the Census Bureau's MCP server offers and writes one request to week 11's API, which we send and check. It starts with its own setup step.

Part 3 — Friday: search by meaning, answer with sources, and tools

On Wednesday the model read one page, and we chose the page. With a hundred passages, we first need a way to find the ones worth reading. An **embedding** turns a passage into a list of numbers, so that passages about the same thing get similar numbers. A question becomes a list of numbers too, and the passages closest to it are the ones to read.

Boulder's reports are a poor test of that. Their pages are mostly tables, and an embedding captures what a passage is about, not which figure it prints: every year's report has a page on marijuana revenue, and to an embedding those pages look nearly alike. Wikipedia suits it better. Its articles are prose, they're rewritten as their subjects' lives change, and every version is kept in the article's history (week 10). So today we take the **lead**, the paragraphs above an article's first heading, of five articles: Barack Obama, Hillary Clinton, Donald Trump, Joe Biden, and Kamala Harris. We take each lead as it stood at midnight UTC on January 1 of every year from 2004 to 2026, and as it stands now. How far was each year's lead from today's, and when did it change most?

Then a model answers a question from the leads, as JSON with the versions it used, or null. Finding the passages is **retrieval**, and writing the answer from them is generation: together, **retrieval-augmented generation**, or **RAG**. Ours keeps the answer to one year and its sources. Last, **tools**. A model can't fetch data, but it can write a request. The Census Bureau's MCP server tells a model which requests it can make of week 11's API; the model chooses one, and we send it and check the answer against week 11's.

Step 11 — Set up, and one old version by hand (7 minutes)

Part 3 starts fresh, and today the setup is given, so we can spend the time on what's new: put your email in its `HEADERS` and run it. It imports what we need, now with NumPy, matplotlib, and BeautifulSoup (week 6), and checks two keys: `OPENROUTER_KEY`, and week 11's `CENSUS_KEY`, which Step

14 needs. Then run the next given cell, whatever the check said: it asks for a key that's `MISSING`, and reads both. Wikimedia asks for week 10's User-Agent, with the library and its version, so `HEADERS` takes that form today. Then get one old version of a lead by hand, in two requests to the Action API. First, `prop=revisions` with `rvstart` set to `2009-01-01T00:00:00Z`, `rvdir=older` and `rvlimit=1` returns one revision: the newest at or before that moment, the version a reader saw on New Year's Day. Its `revid` names that version. Second, `action=parse` with `oldid` set to that `revid`, `prop=text` and `section=0` returns the version's lead as HTML (section 0 is everything above the first heading; `formatversion=2` makes `text` a plain string). Wikipedia keeps the wikitext of old versions, not their HTML, so `parse` draws the 2009 wikitext with today's templates. Templates deleted since then come back as links to missing pages, named `Template:…`. Read each paragraph (`<p>`) with BeautifulSoup's `get_text()`, and look for those leftovers. Then a given cell takes them out, as the chapter's `lead_text()` does: `select()` finds, by CSS selector (week 5), the tables, footnote marks, notes, links to missing templates (`a.new[title^='Template:']`) and error messages (`.error`), and `decompose()` deletes each. Last, a given cell loads all the leads, collected and cleaned the same way on 2026-11-09, and we check that your 2009 lead is the same text as the saved one. See the chapter's "Embeddings and Retrieval".

```
# Given: Part 3's setup. Put your email in HEADERS (week 10's form, with requests' version), then
 ↪  run it.
# It imports what Part 3 needs, checks both keys, sets the three addresses (OpenRouter,
 ↪  Wikipedia's
# Action API, and the course's saved leads). The next cell reads the keys
import os
import re
import json
import time
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": f"WebDataScience/1.0 (your-email@colorado.edu)
 ↪  python-requests/{requests.__version__}"}
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
print("CENSUS_KEY:", "set" if os.environ.get("CENSUS_KEY") else "MISSING")
openrouter_base = "https://openrouter.ai/api/v1"
API = "https://en.wikipedia.org/w/api.php"
wiki =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-13/data/wikipedia/"
```

```
OPENROUTER_KEY: set
CENSUS_KEY: set
```

```
# Given: run this whatever the check said. For a key that was MISSING, it asks with getpass, in a
 ↪  box under this
# cell; then, either way, it reads both keys
from getpass import getpass
for name in ["OPENROUTER_KEY", "CENSUS_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
openrouter_key = os.environ["OPENROUTER_KEY"]
census_key = os.environ["CENSUS_KEY"]
```

```
# params: action=query, titles="Barack Obama", prop=revisions, rvprop="ids|timestamp", rvlimit=1,
#   rvstart="2009-01-01T00:00:00Z", rvdir="older", format=json
# Send it with HEADERS, go down to the page as in week 10, and print its first revision
params = {"action": "query", "titles": "Barack Obama", "prop": "revisions", "rvprop":
 ↪  "ids|timestamp",
```

```
          "rvlimit": 1, "rvstart": "2009-01-01T00:00:00Z", "rvdir": "older", "format": "json"}
data = requests.get(API, params=params, headers=HEADERS).json()
page = list(data["query"]["pages"].values())[0]
revision = page["revisions"][0]
print(revision)
```

```
{'revid': 261157071, 'parentid': 261086351, 'timestamp': '2008-12-31T20:44:09Z'}
```

```
# params: action=parse, oldid=revision["revid"], prop=text, section=0, format=json,
 ↪  formatversion=2; html: the JSON's ["parse"]["text"]
# soup: BeautifulSoup(html, "html.parser"); print the first 150 characters of each <p>'s
 ↪  get_text(" ", strip=True)
params = {"action": "parse", "oldid": revision["revid"], "prop": "text", "section": 0,
          "format": "json", "formatversion": 2}
html = requests.get(API, params=params, headers=HEADERS).json()["parse"]["text"]
soup = BeautifulSoup(html, "html.parser")
for paragraph in soup.find_all("p"):
    print(paragraph.get_text(" ", strip=True)[:150])
```

```
Template:Active editnotice Template:Redirect4 Template:FixHTML
Template:FixHTML
Barack Hussein Obama II ( Template:Pron-en ; born August 4, 1961) is an American politician and
 ↪  the President-elect of the United States . The first A
He is a graduate of Columbia University and Harvard Law School , where he was the first
 ↪  African-American president of the Harvard Law Review . Obama w
As a member of the Democratic minority in the 109th Congress , Obama helped create legislation to
 ↪  control conventional weapons and to promote greater
```

```
# Given: the chapter's lead_text(), as plain lines. decompose() deletes each tag select() finds
 ↪  (tables, footnote
# marks, notes, links to missing templates, error messages); lead joins the text of each <p> that
 ↪  has any, with
# blank lines, and re.sub() takes out the space get_text(" ") leaves before punctuation
for tag in soup.select("table, .reference, .hatnote, .shortdescription, style,
 ↪  a.new[title^='Template:'], .error"):
    tag.decompose()
paragraphs = []
for paragraph in soup.find_all("p"):
    if paragraph.get_text(strip=True):
        paragraphs.append(paragraph.get_text(" ", strip=True))
lead = re.sub(r" ([,.;:])", r"\1", "\n\n".join(paragraphs))
print(lead[:300])
```

```
Barack Hussein Obama II (; born August 4, 1961) is an American politician and the President-elect
 ↪  of the United States. The first African American to be elected President of the United
 ↪  States, Obama was elected to be the junior United States Senator from Illinois in 2004 and
 ↪  served until his resigna
```

```
# Given: every lead, collected and cleaned this way on 2026-11-09: one row per article and year,
 ↪  plus each
# article's "current" version. Collecting them at one request a second drew 429s for about 30
 ↪  minutes
leads = pd.read_csv(wiki + "leads.csv")
print(len(leads), "leads")
print(leads.groupby("title")["year"].count())
```

```
116 leads
title
```

```
Barack Obama       23
Donald Trump       23
Hillary Clinton    24
Joe Biden          24
Kamala Harris      22
Name: year, dtype: int64
```

```
# saved_lead: the "lead" of leads' row for "Barack Obama" and year "2009"; print whether lead ==
 ↪  saved_lead
saved_lead = leads[(leads["title"] == "Barack Obama") & (leads["year"] ==
 ↪  "2009")]["lead"].iloc[0]
print(lead == saved_lead)
```

```
True
```

💬 Discuss

- Collecting these leads at one request a second drew 429 (Too Many Requests) for about 30 minutes, with waits of up to 40 seconds. If all of us ran that loop at once from this room, whose requests would Wikipedia refuse, and why does week 10's User-Agent matter here?
- Our two requests assume they worked. Wikimedia's 429 is plain text, not JSON (week 10): which line would fail, and what would it say?
- `parse` drew the 2009 version with today's templates, and our cleanup took out what it could see was broken. What might still come from today rather than from 2009, without looking broken?

Step 12 — Embed, and compare each year with today (8 minutes)

OpenRouter's embeddings endpoint takes a `model` and an `input`, a list of texts. It returns one **vector** per text: the list of numbers that is the text's embedding. We use one free model, `EMBEDDER`, for every vector. Two models turn the same text into different numbers, so their vectors can't be compared, and an embedding model never gets a fallback list. It reads at most about 4,096 tokens of a text, which is why we embed the lead and not the article: on January 1, 2009, Barack Obama's whole article was already 135,666 bytes of wikitext. Embed one lead by hand first, and count its numbers. Then the given cell embeds all 116 leads, 20 to a request, and scales each vector to length 1. (A vector's length here is the square root of its squared numbers added up, not how many numbers it has.) If the free model's daily limit stops that cell, the next one loads the same vectors, saved on 2026-11-09.

To compare two leads we use **cosine similarity**, which measures how nearly two vectors point the same way: 1 if they point exactly the same way, less the further apart they point. For vectors of length 1, it's their **dot product**, `@`: multiply the two lists number by number, and add up. Compare each lead with its article's current lead. Then `pivot()` the result into a table with a row per year and a column per article; a given cell plots it. See the chapter's "Embedding and Cosine Similarity".

```
# EMBEDDER = "nvidia/nemotron-3-embed-1b:free"; one: the JSON of a POST to
 ↪  f"{openrouter_base}/embeddings"
# with json {"model": EMBEDDER, "input": [leads["lead"][0]]}; print how many numbers its first
 ↪  embedding holds, and its "usage"
EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
one = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
 ↪  [leads["lead"][0]]},
                    headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(len(one["data"][0]["embedding"]), "numbers")
print(one["usage"])
```

```
2048 numbers
{'prompt_tokens': 205, 'total_tokens': 205, 'cost': 0, 'is_byok': False, 'cost_details':
 ↪  {'upstream_inference_cost': 0, 'upstream_inference_prompt_cost': 0,
 ↪  'upstream_inference_completions_cost': 0}}
```

```
# Given: embed every lead, 20 to a request, with a pause between requests; vectors: one row per
 ↪  lead, scaled to length 1
vectors = []
for start in range(0, len(leads), 20):
    batch = leads["lead"][start:start + 20].tolist()
    response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
 ↪  batch},
                             headers={"Authorization": f"Bearer {openrouter_key}",
 ↪  **HEADERS}).json()
    vectors += [item["embedding"] for item in response["data"]]
    time.sleep(1)
vectors = np.array(vectors)
vectors = vectors / np.linalg.norm(vectors, axis=1, keepdims=True)
print(vectors.shape)
```

```
(116, 2048)
```

```
# Given, only if the cell above stopped: remove the "#" from the next two lines, and run this
 ↪  cell. It loads the
# vectors saved on 2026-11-09, from the same model, already scaled to length 1, in the same order
 ↪  as leads
# import io
# vectors = np.load(io.BytesIO(requests.get(wiki + "lead-vectors.npy", headers=HEADERS).content))
```

```
# today: a dictionary from each title to the vector of its "current" lead (row i of vectors is
 ↪  row i of leads)
# leads["similarity"]: each lead's vector @ today[its title]; print the current rows' similarity,
 ↪  which should be 1
today = {}
for i, row in leads.iterrows():
    if row["year"] == "current":
        today[row["title"]] = vectors[i]
leads["similarity"] = [vectors[i] @ today[title] for i, title in enumerate(leads["title"])]
print(leads[leads["year"] == "current"][["title", "similarity"]])
```

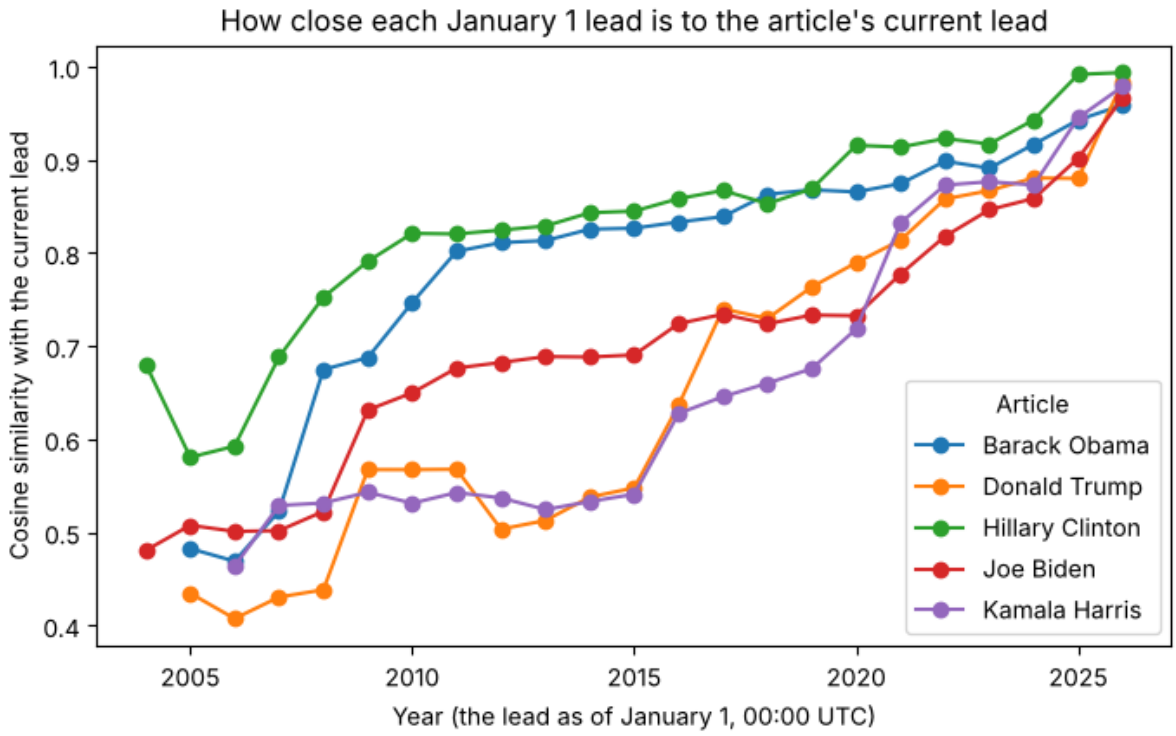
```
              title  similarity
22      Barack Obama         1.0
46   Hillary Clinton         1.0
69      Donald Trump         1.0
93         Joe Biden         1.0
115    Kamala Harris         1.0
```

```
# table: the leads whose year isn't "current", pivoted to a row per year and a column per title,
 ↪  with similarity as values
# Make its index (the years) integers; print it, rounded to 3 places
table = leads[leads["year"] != "current"].pivot(index="year", columns="title",
 ↪  values="similarity")
table.index = table.index.astype(int)
print(table.round(3))
```

```
title  Barack Obama  Donald Trump  Hillary Clinton  Joe Biden  Kamala Harris
year
2004            NaN           NaN            0.680      0.481            NaN
2005          0.482         0.435            0.581      0.508            NaN
2006          0.469         0.407            0.592      0.501          0.464
2007          0.523         0.430            0.688      0.502          0.529
2008          0.675         0.438            0.752      0.522          0.531
```

2009	0.687	0.567	0.791	0.631	0.543
2010	0.747	0.567	0.821	0.650	0.531
2011	0.802	0.568	0.820	0.676	0.543
2012	0.811	0.503	0.824	0.682	0.537
2013	0.813	0.513	0.829	0.689	0.524
2014	0.825	0.538	0.843	0.688	0.533
2015	0.826	0.548	0.845	0.690	0.541
2016	0.833	0.637	0.858	0.724	0.628
2017	0.839	0.740	0.867	0.734	0.646
2018	0.863	0.729	0.853	0.724	0.660
2019	0.868	0.763	0.869	0.733	0.676
2020	0.865	0.789	0.915	0.732	0.718
2021	0.874	0.814	0.913	0.777	0.832
2022	0.898	0.858	0.923	0.818	0.873
2023	0.891	0.867	0.917	0.846	0.876
2024	0.917	0.880	0.943	0.858	0.873
2025	0.943	0.880	0.992	0.901	0.945
2026	0.959	0.983	0.993	0.966	0.979

```
# Given: plot table, a line per article with a marker at each year, titled, with both axes and
↪  the legend labeled
fig, ax = plt.subplots(figsize=(8, 4.5))
table.plot(ax=ax, marker="o")
ax.set_title("How close each January 1 lead is to the article's current lead")
ax.set_xlabel("Year (the lead as of January 1, 00:00 UTC)")
ax.set_ylabel("Cosine similarity with the current lead")
ax.legend(title="Article")
plt.show()
```



⏱️ Microsprint (4 minutes, alone or in pairs)

Each line climbs toward 1, but not evenly. A jump is a year whose lead moved much closer to today's than the year before's had.

Do: pick one article. `diff()` gives each year's change from the year before, and `idxmax()` the year of the biggest rise. Print that year and the size of its jump; then print the first 400 characters of that year's lead and of the year before's. **Share:** the article, the year, the jump, and what the lead said differently.

```
# Microsprint: title: your article; jumps: table[title].diff(); year: jumps.idxmax(); print year
 ↪  and the jump, rounded to 3
# Then for the year before and that year, print the first 400 characters of the lead (leads'
 ↪  "year" holds text: use str())
title = "Kamala Harris"
jumps = table[title].diff()
year = jumps.idxmax()
print(year, round(jumps.max(), 3))
for y in [year - 1, year]:
    print(y, leads[(leads["title"] == title) & (leads["year"] == str(y))]["lead"].iloc[0][:400],
     ↪  "\n")
```

```
2021 0.114
2020 Kamala Devi Harris ( / ˈ k ɑː m ə l ə / KAH -mə-lə; born October 20, 1964) is an American
 ↪  lawyer and politician serving as the junior United States Senator from California since 2017.
 ↪  A member of the Democratic Party, she previously served as the 27th District Attorney of San
 ↪  Francisco from 2004 to 2011 and 32nd Attorney General of California from 2011 until 2017. She
 ↪  ran as a candidate for the De

2021 Kamala Devi Harris ( / ˈ k ɑː m ə l ə / KAH -mə-lə; born October 20, 1964) is an American
 ↪  politician who is the vice president-elect of the United States and the junior United States
 ↪  senator from California. Prior to her election to the Senate, she served as the Attorney
 ↪  General of California.

A member of the Democratic Party, she will become vice president on January 20, 2021, alongside
 ↪  Presiden
```

💬 Discuss

- Harris's lead jumped most in 2021 (0.114), when it first called her vice president-elect, and Biden's in 2009, for the same reason. Trump's biggest jump came in 2009, not 2017: his lead grew from 93 words to 353 (the `words` column), with no new office. What besides events can move a lead closer to today's?
- Clinton's lead went from 0.943 in 2024 to 0.992 in 2025, with no new office: editors rewrote her opening. What does a similarity measure, and which claims about how Wikipedia described these people can it not support?
- The given loop assumes every request worked. If the free model answered 429 to the fourth batch, the cell would stop at `response["data"]`: what would `vectors` hold, and what does the saved vectors' cell spare you?

Step 13 — Answer with sources, as JSON (9 minutes)

Now the generation half of RAG. Our question is about when: *In which year did the article first say Joe Biden is the 46th president of the United States?* We'll ask more than one question, so first gather Step 12's lines into a function, `search(question, title)`. It embeds the question the way we embedded a lead, scales its vector to length 1, scores every lead against it with `@`, and returns the three best-scoring leads of one article. Those are the **passages**, each headed by its ID, such as `Joe`

`Biden 2024`. Send them with the question and a schema whose `year` may be **null**, JSON's word for "no value" (`None` in Python): a null is the model's way to say the passages don't say. The request is Wednesday's, with a new instruction and schema. Then check the answer the way week 9 checked pdfplumber's: against the source, by reading the leads. See the chapter's "Retrieval-Augmented Extraction".

```
# Given: each lead's ID, such as "Joe Biden 2024"; and the shape of an answer with its sources.
 ↪  year may be null
leads["id"] = leads["title"] + " " + leads["year"]
ANSWER_SCHEMA = {
    "type": "object",
    "properties": {
        "year": {"type": ["integer", "null"],
                 "description": "The earliest year whose passage states the fact; null if no
                  ↪  passage states it"},
        "source_ids": {"type": "array", "items": {"type": "string"},
                       "description": "The IDs of the passages that state it; empty if year is
                        ↪  null"},
    },
    "required": ["year", "source_ids"],
    "additionalProperties": False,
}
ANSWER_INSTRUCTION = ("Answer the question using only the passages, each headed by its ID. "
                      "A passage is one year's version of a Wikipedia article's lead. "
                      "Give a year only if a passage states the fact; if none does, answer null.
                       ↪  "
                      "Answer with JSON that fits this schema: " + json.dumps(ANSWER_SCHEMA))
```

```
# search(question, title): Step 12's lines, gathered: embed the question, scale its vector to
 ↪  length 1,
# put vectors @ it in leads["score"], and return the three best-scoring leads of that title
def search(question, title):
    response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
 ↪  [question]},
                             headers={"Authorization": f"Bearer {openrouter_key}",
 ↪  **HEADERS}).json()
    query = np.array(response["data"][0]["embedding"])
    leads["score"] = vectors @ (query / np.linalg.norm(query))
    return leads[leads["title"] == title].nlargest(3, "score")
```

```
# question: "In which year did the article first say Joe Biden is the 46th president of the
 ↪  United States?"
# top: search(question, "Joe Biden"); print its IDs and scores; passages: each "ID: lead", joined
 ↪  with blank lines
question = "In which year did the article first say Joe Biden is the 46th president of the United
 ↪  States?"
top = search(question, "Joe Biden")
print(top[["id", "score"]])
passages = "\n\n".join(f"{row.id}: {row.lead}" for row in top.itertuples())
```

```
                id     score
92     Joe Biden 2026  0.449356
90     Joe Biden 2024  0.448661
93  Joe Biden current  0.446402
```

```
# body: Wednesday's, with ANSWER_INSTRUCTION, the user message f"Question:
 ↪  {question}\n\n{passages}", ANSWER_SCHEMA named "answer",
```

```
# and provider {"require_parameters": True}, as on Wednesday. Send it; print the model that
 ↪  answered and its answer
MODELS = ["nvidia/nemotron-3.5-super-vl:free", "google/gemma-4-31b-it:free",
 ↪  "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
        "messages": [{"role": "system", "content": ANSWER_INSTRUCTION},
                     {"role": "user", "content": f"Question: {question}\n\n{passages}"}],
        "response_format": {"type": "json_schema", "json_schema": {"name": "answer", "strict":
         ↪  True, "schema": ANSWER_SCHEMA}},
        "provider": {"require_parameters": True}}
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
```

```
nvidia/nemotron-3.5-super-vl:free {'year': 2024, 'source_ids': ['Joe Biden 2026', 'Joe Biden
 ↪  2024', 'Joe Biden current']}
```

```
# Check it by reading: for each Joe Biden lead from 2020 on (leads["year"] >= "2020" keeps
 ↪  "current" too),
# print its ID and its first 300 characters. In which year's lead is he the 46th president, not
 ↪  about to be?
for row in leads[(leads["title"] == "Joe Biden") & (leads["year"] >= "2020")].itertuples():
    print(row.id, "|", row.lead[:300], "\n")
```

```
Joe Biden 2020 | Joseph Robinette Biden Jr. ( / ˌ r ɒ b ɪ ˈ n ɛ t ˈ b aɪ d ən /; born November
 ↪  20, 1942) is an American politician who served as the 47th vice president of the United
 ↪  States from 2009 to 2017. Biden also represented Delaware in the U.S. Senate from 1973 to
 ↪  2009. A member of the Democratic Party, Bide

Joe Biden 2021 | Joseph Robinette Biden Jr. ( / ˈ b aɪ d ən / BY -dən; born November 20, 1942) is
 ↪  an American politician and the president-elect of the United States. Biden defeated incumbent
 ↪  president Donald Trump in the 2020 presidential election and will be inaugurated as the 46th
 ↪  president on January 20, 2021. A

Joe Biden 2022 | Joseph Robinette Biden Jr. ( / ˈ b aɪ d ən / BY -dən; born November 20, 1942) is
 ↪  an American politician who is the 46th and current president of the United States. A member
 ↪  of the Democratic Party, he served as the 47th vice president from 2009 to 2017 under Barack
 ↪  Obama and represented Delaware in

Joe Biden 2023 | Joseph Robinette Biden Jr. ( / ˈ b aɪ d ən / ⓘ BY -dən; born November 20, 1942)
 ↪  is an American politician who is the 46th and current president of the United States. A
 ↪  member of the Democratic Party, he previously served as the 47th vice president from 2009 to
 ↪  2017 under President Barack Obama, and

Joe Biden 2024 | Joseph Robinette Biden Jr. ( / ˈ b aɪ d ən / ⓘ BY -dən; born November 20, 1942)
 ↪  is an American politician who is the 46th and current president of the United States. A
 ↪  member of the Democratic Party, he previously served as the 47th vice president from 2009 to
 ↪  2017 under President Barack Obama and r

Joe Biden 2025 | Joseph Robinette Biden Jr. ( / ˈ r ɒ b ɪ n ɪ t ˈ b aɪ d ən / ⓘ ROB -in-it BY
 ↪  -dən; born November 20, 1942) is an American politician who has been the 46th and current
 ↪  president of the United States since 2021. A member of the Democratic Party, he served as the
 ↪  47th vice president from 2009 to 2017 u
```

```
Joe Biden 2026 | Joseph Robinette Biden Jr. (born November 20, 1942) is an American politician
 ↪  who was the 46th president of the United States from 2021 to 2025. As a member of the
 ↪  Democratic Party, he represented Delaware in the United States Senate from 1973 to 2009.
 ↪  Biden served as the 47th vice president under P

Joe Biden current | Joseph Robinette Biden Jr. (born November 20, 1942) is an American retired
 ↪  politician who was the 46th president of the United States from 2021 to 2025. A member of the
 ↪  Democratic Party, he represented Delaware in the United States Senate from 1973 to 2009 and
 ↪  also served as the 47th vice president
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: ask "In which year did the article first say Kamala Harris is the 47th president of the United States?" the same way: `search()` Kamala Harris's leads, and send the three best in `body`. **Share:** the three IDs, and the model's answer. Is it right, and how do you know?

```
# Microsprint: set question; top: search(question, "Kamala Harris"); print its IDs; rebuild
 ↪  passages,
# put them in body's user message (body["messages"][1]["content"]), and send it; print the model
 ↪  and its answer
question = "In which year did the article first say Kamala Harris is the 47th president of the
 ↪  United States?"
top = search(question, "Kamala Harris")
print(top[["id", "score"]])
passages = "\n\n".join(f"{row.id}: {row.lead}" for row in top.itertuples())
body["messages"][1]["content"] = f"Question: {question}\n\n{passages}"
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
```

```
                      id     score
111  Kamala Harris 2023  0.426085
110  Kamala Harris 2022  0.423550
112  Kamala Harris 2024  0.423322
nvidia/nemotron-3.5-super-vl:free {'year': None, 'source_ids': []}
```

💬 Discuss

- The model answered 2024 from the leads of 2026, 2024 and today, and 2024 is the earliest of those three. Reading shows 2022. Why can't a ranking by similarity answer a question about the *first* time, and what would you send the model instead?
- A text search, `leads["lead"].str.contains("46th president")`, finds Biden's 2021, 2026 and current leads, and misses 2022 to 2025, which say "the 46th and current president". Its 2021 hit says he "will be inaugurated", and Trump's 2025 lead says the same of the 47th. What does reading catch that a search can't?
- For Harris the model answered null from three of her 22 leads. Does that show that no lead ever called her president? What check would?

Step 14 — Tools: a model writes a Census request, and skills (8 minutes)

A model can't fetch data: all it can do is write text. But a request is text, so if a program tells a model which requests it can make, the model can write one, and the program sends it. Each request a model may make is a **tool**: a name, a description, and its inputs, described by a JSON Schema like Wednesday's. The **Model Context Protocol** (MCP) is a shared way for a server to tell a program which tools it offers: the program sends the method `tools/list`, and the server answers with each tool's `name`, `description`, and `inputSchema`. The Census Bureau's MCP server offers five tools for week

11's API. It runs on your own computer, with Docker or Node, so we don't install it: it was run once, on 2026-11-09, and its answers were saved in this course's repository, in `labs/week-13/data/census-mcp/`. Two given cells read its tools from there and put them in the form OpenRouter's tool calling asks for: each tool's `inputSchema` becomes its `parameters`. We send them to a model with a week 11 question, and `tool_choice` `"auto"`, OpenRouter's default, lets the model choose between a tool call and text. The chapter sends `"required"`, OpenAI's value for a call and no text, which OpenRouter's documentation doesn't list; on 2026-11-09 Nemotron honored it, but Dots, the last of `MODELS`, answered it with garbled arguments or no call at all, and with `"auto"` it wrote the right call three times in five. With `"auto"`, a model may answer in prose instead, with no `tool_calls`; when it does call a tool, the call comes back in the message's `tool_calls`, with its `arguments` as a JSON string. The model only writes the call. A given cell runs it, as the server would have: it reads the call's arguments with `json.loads()`, prints whether the model chose `fetch-aggregate-data`, the one tool our code runs, as the chapter checks the call before running it; then turns the arguments into week 11's request, sends it with `census_key`, and prints the answer, never the URL, which holds your key. We check that answer against week 11's figure, $103,994 with a margin of error of $2,268, and against what the real server returned. Last, a **skill**, plainer still: a folder of instructions in Markdown, whose `name` and `description` tell a model when to read the rest. A given cell reads the first lines of a public one, for PDFs, and its headings. See the chapter's "Tools, MCP, and Skills", and its "Running the Call Yourself" for the full check.

```
# Given: the Census MCP server's answer to tools/list, saved in the course's repository; print
 ↪  each tool's name
# and the inputs its JSON Schema requires
cmcp =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-13/data/census-mcp/"
tools = requests.get(cmcp + "tools.json", headers=HEADERS).json()
for tool in tools:
    print(tool["name"], tool["inputSchema"]["required"])
```

```
fetch-aggregate-data ['dataset', 'year', 'get']
fetch-dataset-geography ['dataset']
list-datasets []
resolve-geography-fips ['geography_name']
search-data-tables []
```

```
# Given: each MCP tool in OpenRouter's form, its inputSchema as "parameters"; the instruction;
 ↪  and week 11's question,
# with the FIPS codes week 11 used
functions = [{"type": "function", "function": {"name": tool["name"], "description":
 ↪  tool["description"],
                                               "parameters": tool["inputSchema"]}} for tool in
                                                ↪  tools]
TOOL_INSTRUCTION = "Choose one tool call that answers the question. Use the tools' parameters
 ↪  exactly."
question = ("What was the median household income of Boulder County, Colorado (state FIPS 08,
 ↪  county FIPS 013), "
            "in the 2024 ACS 5-year estimates, table B19013?")
```

```
# body: MODELS, temperature 0, "tools": functions, "tool_choice": "auto", messages (system
 ↪  TOOL_INSTRUCTION, user question)
# Send it as on Wednesday; call: the first choice's message's "tool_calls"[0]["function"]
# Print the model that answered, call["name"], and call["arguments"], a string of JSON
body = {"models": MODELS, "temperature": 0, "tools": functions, "tool_choice": "auto",
        "messages": [{"role": "system", "content": TOOL_INSTRUCTION}, {"role": "user", "content":
         ↪  question}]}
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
call = response["choices"][0]["message"]["tool_calls"][0]["function"]
```

```
print(response["model"], call["name"], call["arguments"])
```

```
nvidia/nemotron-3.5-super-vl:free fetch-aggregate-data {"dataset":"acs/acs5","year":2024,"get":{"group":"B19013"},"for":"county:013","in":"state:08"}
```

```
# Given: run the call ourselves. args: the call's arguments, from JSON to a dictionary, read here so that this cell
# always runs the call it checks. Is it the one tool this code runs? Then url: week 11's address, from args' year
# and dataset; get: group(...) for a whole table, or the variables joined with commas; send it with census_key,
# and print data, never the URL: your key is in it
args = json.loads(call["arguments"])
print(call["name"], call["name"] == "fetch-aggregate-data")
url = f"https://api.census.gov/data/{args['year']}/{args['dataset']}"
if "group" in args["get"]:
    get = f"group({args['get']['group']})"
else:
    get = ",".join(args["get"]["variables"])
params = {"get": get, "for": args["for"], "in": args["in"], "key": census_key}
data = requests.get(url, params=params, headers=HEADERS).json()
print(data)
```

```
fetch-aggregate-data True
```

```
[['B19013_001E', 'B19013_001EA', 'B19013_001M', 'B19013_001MA', 'GEO_ID', 'NAME', 'state', 'county'], ['103994', None, '2268', None, '0500000US08013', 'Boulder County, Colorado', '08', '013']]
```

```
# row: dict(zip()) of data's names (row 0) and values (row 1); print its B19013_001E, whether it's week 11's "103994",
# and its margin of error, row.get("B19013_001M"); then print the text of session.json's last message, the real server's answer
row = dict(zip(data[0], data[1]))
print(row["B19013_001E"], row["B19013_001E"] == "103994", row.get("B19013_001M"))
session = requests.get(cmcp + "session.json", headers=HEADERS).json()
print(session[-1]["received"]["result"]["content"][0]["text"])
```

```
103994 True 2268
```

```
Response from acs/acs5:
NAME: Boulder County, Colorado, B19013_001E: 103994, B19013_001M: 2268, state: 08, county: 013
Source: U.S. Census Bureau Data API (https://api.census.gov/data/2024/acs/acs5?get=NAME%2CB19013_001E%2CB19013_001M&for=county%3A013&in=state%3A08&descriptive=false&key=REDACTED)
```

```
# Given: a public skill's SKILL.md, for PDFs: its first 5 lines (the front matter: name and description),
# then its "### " headings, the tools it teaches
skill = requests.get("https://raw.githubusercontent.com/anthropics/skills/main/skills/pdf/SKILL.md", headers=HEADERS).text
print("\n".join(skill.splitlines()[:5]))
print([line for line in skill.splitlines() if line.startswith("### ")])
```

```
---
name: pdf
```

```
description: Use this skill whenever the user wants to do anything with PDF files. This includes
 ↪  reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one,
 ↪  splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF
 ↪  forms, encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to make them
 ↪  searchable. If the user mentions a .pdf file or asks to produce one, use this skill.
license: Complete terms in LICENSE.txt
---
['### pypdf - Basic Operations', '### pdfplumber - Text and Table Extraction', '### reportlab -
 ↪  Create PDFs', '### pdftotext (poppler-utils)', '### qpdf', '### pdftk (if available)', '###
 ↪  Extract Text from Scanned PDFs', '### Add Watermark', '### Extract Images', '### Password
 ↪  Protection']
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: ask as a person would, by name and without FIPS codes: "What was the median household income of the city of Boulder, Colorado, in the 2024 ACS 5-year estimates, table B19013?" Put it in `body`'s user message, send it, and print the tool call. **Share:** the tool the model chose, and its arguments. Could our cell that runs the call run this one? What would you send instead, with week 11's place code for Boulder, 07850?

```
# Microsprint: put the question above in body's user message (body["messages"][1]["content"]),
 ↪  and send it
# call: the first tool call's "function", as above; print the model, call["name"], and
 ↪  json.loads() of its "arguments"
body["messages"][1]["content"] = ("What was the median household income of the city of Boulder,
 ↪  Colorado, "
                                  "in the 2024 ACS 5-year estimates, table B19013?")
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
call = response["choices"][0]["message"]["tool_calls"][0]["function"]
print(response["model"], call["name"], json.loads(call["arguments"]))
```

```
nvidia/nemotron-3.5-super-vl:free resolve-geography-fips {'geography_name': 'Boulder city,
 ↪  Colorado', 'summary_level': 'Place'}
```

💬 Discuss

- In the key's run, the microsprint's model chose `resolve-geography-fips`, a tool we haven't written, which the real server answers from a database it builds when it's installed. What would our cell that runs the call do with it, and what should a program do with a call it can't run?
- Our cell checked the tool's name, but the model's arguments went into our request unchecked. In an earlier run the model also sent `descriptive`, and our cell quietly dropped it. Who should check them before they're sent: the model, the schema, our code, or the Census API? In week 11 the API answered 400 to a place without a state.
- MCP standardizes how tools are described, so any model can read the Census server's. It doesn't make the answer right: what did we check ours against? In the key's run the model asked for the whole table, `group` B19013, margin of error included; in an earlier run it asked for the estimate alone, `B19013_001E`, and the margin printed `None`. Would we have noticed the missing margin without week 11's figure?

Step 15 — What did we learn? (3 minutes)

In a few sentences, as comments:

- Datalab read a picture of a table and wrote descriptions of charts. Which of its output would you publish, and which would you check first?
- On Wednesday, which reports did the model get wrong, and was the problem the model or what we sent it?

- Step 12 put a number on how far each year's lead was from today's. Which of its jumps could you explain only by reading the leads, and what can a similarity not tell you?
- Every call this week returned JSON, and every answer was checked: against week 9's figures, a sum, the leads themselves, or week 11's figure. What would you check your own project's model outputs against?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, convert a public PDF you choose, with no private individuals' names, and save its Markdown: Monday's Steps 0–3 and 5, the lines the chapter's `convert()` gathers. For a PDF on your computer rather than on the web, the chapter's "Submitting a Document" says how to upload it.
- Step 2, a schema of your own, and an instruction that includes it: Step 8's `SCHEMA`, and Step 10's `INSTRUCTION` with the schema added.
- Step 3, send the same request three times: Step 8's request, `require_parameters` included, and Step 10's pause between requests.
- Step 4, your own ground truth, and one check on a table: Step 9's comparison with week 9, and Step 4's sums.
- Step 5, six leads of an article you choose, their similarity to the current lead, and one "first year" question: Friday's Step 11, the two requests and the cleanup the chapter's `lead_text()` gathers; Step 12's embeddings and similarities; and Step 13's `search()`, answer with sources, and check by reading.
- Step 6, report the costs, the models that answered, and which check caught each wrong answer: Step 2's `page_count` and `total_cost`, each response's `model` and `usage`, and the checks of Steps 9, 10 and 13.

```
# Your answers here, as comments
# 1. Exhibit 3's table: all 30 rows passed week 9's sums (and 2018's 28), so once a person had
 ↪  checked its labels
#    against the image it could be the screen readable version. Never the chart descriptions:
 ↪  Chart 8's calls
#    $1,262,688 the largest slice and then names $1,341,007.
# 2. In the key's run, both totals matched week 9 for 4 of 7 reports, and all four fields for
 ↪  only 2 (2022, 2023).
#    For 2018 and 2019 it gave the subtotal "based on current economic activity" as the year's
 ↪  total; for 2021 it returned
#    1,000,000 and 1,200,000, which aren't on the page; it gave January for 2019-2021, from
 ↪  "received by the city
#    through January ...", and four years were wrong. Each page held the right row. With the
 ↪  schema added to the
#    instruction, the same loop got all four fields right for 7 of 7, so the problem was what we
 ↪  sent.
# 3. Harris's 2021 (+0.114) and Biden's 2009 (+0.109) jumps came with an election, but Trump's
 ↪  biggest, 2009 (+0.129),
#    was his lead growing from 93 words to 353, and Clinton's 2025 (+0.049) was a rewrite. A
 ↪  similarity says how much
#    the words moved toward today's, not why, or whether the lead was right.
# 4. Week 9's figures, a sum, the leads themselves (the model answered 2024 for Biden; reading
 ↪  gave 2022), and
#    week 11's 103,994 and 2,268 for the Census call the model wrote. For a project: a hand-coded
 ↪  sample, a published
#    table, or a second tool.
```

Friday: your portfolio checkpoint (15 minutes)

The last 15 minutes go to your project. This week's checkpoint, **Analyze your data**, is due Sunday, November 15:

- **An analysis notebook** that runs top to bottom, with its output visible;
- **results that answer at least part of your question**: the counts or comparisons your proposal named;
- **at least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption;
- **the README's Methods section** and your preliminary **Findings**.

Use the time to run your analysis notebook once, top to bottom, here, where you can ask. If you use a model anywhere in your analysis, say which, when, and what you checked its output against, in Methods.